

AI 学习小组 | 第一课：计算机基础五问

从可编程机器到 AI：历史、硬件、语言、操作系统与“理解”

01 历史 | 这些人分别解决了什么问题？

巴贝奇／洛芙莱斯 19 世纪	巴贝奇设计差分机与可编程分析机；阿达在 1843 年的注释中给出计算伯努利数的算法，并预见机器能处理数字以外的符号。分析机当时未完整建成。
图灵 1936	提出图灵机与通用机器的理论模型，回答“什么可以被计算”，并揭示计算的边界；也参与二战密码分析。
冯·诺伊曼 1945	在 EDVAC 报告中系统阐述存储程序设计：程序和数据存入存储器，由处理器取指执行。这是多人共同发展的成果。
香农 1937／1948	将布尔逻辑用于开关电路设计；建立信息论，用熵等概念研究信息量、编码与通信极限。[1]
霍珀 20 世纪 50 年代	开发早期编译系统 A-0，推动接近英语的编程方式，并参与推动 COBOL 发展，让编程更接近人的表达。
汤普森／里奇 1969 年起	汤普森主导早期 Unix 并创造 B 语言；里奇创造 C 语言，二人共同发展 Unix，深刻影响操作系统与软件移植。[2]

发展主线：机械计算 → 电子计算 → 晶体管与集成电路 → 微处理器与个人电脑 → 网络与云计算 → 大规模并行计算与 AI。

02 硬件 | 计算机靠什么工作？

基本功能：运算器＋控制器（通常组成 CPU）、存储器、输入／输出，以及连接它们的电路与供电。常见电脑还有内存、SSD、主板等；独立显卡、显示器并非所有计算机都必需。

CPU：中央处理器	擅长复杂逻辑、分支判断和任务调度，注重单个任务的响应；也能并行计算。
GPU：图形处理器	擅长同时对大量数据执行相似运算，注重总吞吐量；并非所有任务都更快。[3]

为什么适合 AI 训练？神经网络训练反复进行矩阵乘法与梯度计算，可拆成大量并行运算；GPU 的计算单元、内存带宽及专用矩阵单元有利于加速，CPU 仍负责组织任务。[3]

03 编程语言 | 为何不能只靠一句人话？

自然语言可作为入口，但硬件不能直接执行它。自然语言常有歧义、条件遗漏；编程语言用明确规则描述操作，由编译器／解释器等落实为机器指令。AI 能辅助转换，但结果需要检查。

化学例子：“把这组光谱处理一下”不够明确；需说明波长范围、扣背景方法、是否归一化、输出格式。这些要求即使交给 AI，也应写清楚。

为什么有很多语言？不同任务在效率、易用性、安全性和历史兼容上取舍不同：Python 便于数据分析与 AI；C/C++ 便于底层控制和高性能计算；JavaScript 服务网页交互；SQL 用于查询数据库。

04 操作系统 | 谁来管理硬件与程序？

操作系统统一管理处理器、内存、文件、设备与权限，并为程序提供公共接口，避免每个程序从头控制硬件。简单嵌入式程序也可以不使用完整操作系统。

Windows	微软主导的专有操作系统，桌面商业软件与外设生态丰富，常用于办公及仪器控制。
Linux	严格说是开源内核；Ubuntu 等发行版配套软件形成完整系统。便于定制、自动化与服务器部署；也能使用图形界面。

为什么换台电脑就跑不了？系统接口、CPU 架构、语言和软件库版本、驱动、路径或权限可能不同。复现应同时记录代码、数据和环境；虚拟环境／容器能减少差异，但不能消除所有硬件限制。

05 本质 | 计算机在“处理”，还是在“理解”？

与普通固定功能工具的关键区别：通用计算机可通过更换程序，自动完成不同的信息处理任务；同一硬件可用于写作、绘图或分子计算，其能力仍受算法与资源限制。

“处理”是明确机制，“理解”取决于定义。底层始终在执行运算；AI 可学习表征并表现出归纳和推理能力，但这本身不能证明它具有人的意识、体验或对现实的完整认识。

它“理解”化学吗？它可以借助模型和数据预测性质、提出反应方案；量化软件也可按物理模型求解。但会输出机理图，不代表机理已被证实。评价应看证据、适用范围、预测与实验能否相符，而非语言是否流畅。

[1] Bell Labs: Claude Shannon

[2] D. Ritchie: The Development of C

[3] NVIDIA: CUDA Introduction